

[Skip to main content](#)

Back to top ☐ ☐

Search the docs ... Ctrl+K



[PyTorch - Home](#)

[PyTorch - Home](#)

- [Install PyTorch](#)
- [User Guide](#)
 - [PyTorch Main Components](#)
 - [torch.compiler](#)
 - [torch.export](#)
 - [Developer Notes](#)
 - [Accelerator Integration](#)
- [Reference API](#)
 - [torch](#)
 - [torch.nn](#)
 - [torch.nn.functional](#)
 - [torch.Tensor](#)
 - [Tensor Attributes](#)
 - [Tensor Views](#)
 - [Automatic Mixed Precision package - torch.amp](#)
 - [Automatic differentiation package - torch.autograd](#)
 - [torch.library](#)
 - [torch.accelerator](#)
 - [torch.cpu](#)
 - [torch.cuda](#)
 - [Understanding CUDA Memory Usage](#)
 - [torch.mps](#)
 - [torch.xpu](#)
 - [torch.mtia](#)
 - [torch.mtia.memory](#)
 - [torch.mtia.mtia_graph](#)
 - [Meta device](#)
 - [torch.backends](#)
 - [torch.export](#)
 - [Distributed communication package - torch.distributed](#)
 - [torch.distributed.tensor](#)
 - [Generic Join Context Manager](#)
 - [Torch Distributed Elastic](#)
 - [FullyShardedDataParallel](#)
 - [torch.distributed.fsdp.fully_shard](#)
 - [Tensor Parallelism - torch.distributed.tensor.parallel](#)
 - [Distributed Optimizers](#)
 - [Pipeline Parallelism](#)
 - [PyTorch Symmetric Memory](#)
 - [Distributed Checkpoint - torch.distributed.checkpoint](#)
 - [Probability distributions - torch.distributions](#)
 - [torch.compiler API reference](#)
 - [torch.fft](#)
 - [torch.func](#)
 - [torch.futures](#)
 - [torch.fx](#)
 - [torch.fx.experimental](#)
 - [torch.hub](#)
 - [torch.linalg](#)
 - [torch.monitor](#)
 - [torch.signal](#)
 - [torch.special](#)
 - [torch.overrides](#)
 - [torch.nativert](#)
 - [torch.package](#)
 - [torch.profiler](#)
 - [torch.nn.init](#)
 - [torch.nn.attention](#)

- [torch.onnx](#)
- [torch.optim](#)
- [Complex Numbers](#)
- [DDP Communication Hooks](#)
- [Quantization](#)
- [Distributed RPC Framework](#)
- [torch.random](#)
- [torch.masked](#)
- [torch.nested](#)
- [torch.Size](#)
- [torch.sparse](#)
- [torch.Storage](#)
- [torch.testing](#)
- [torch.utils](#)
- [Benchmark Utils - torch.utils.benchmark](#)
- [torch.utils.checkpoint](#)
- [torch.utils.cpp_extension](#)
- [torch.utils.data](#)
- [torch.utils.deterministic](#)
- [JIT Utils - torch.utils.jit](#)
- [torch.utils.dlpack](#)
- [torch.utils.mobile_optimizer](#)
- [torch.utils.model_zoo](#)
- [torch.utils.tensorboard](#)
- [torch.utils.module_tracker](#)
- [Type Info](#)
- [torch.config](#)
- [torch.__future__](#)
- [torch._logging](#)
- [Torch Environment Variables](#)
- [Developer Notes](#)
 - [Automatic Mixed Precision examples](#)
 - [Autograd mechanics](#)
 - [Broadcasting semantics](#)
 - [CPU threading and TorchScript inference](#)
 - [CUDA semantics](#)
 - [PyTorch Custom Operators Landing Page](#)
 - [Distributed Data Parallel](#)
 - [Extending PyTorch](#)
 - [Extending torch.func with autograd.Function](#)
 - [Frequently Asked Questions](#)
 - [Getting Started on Intel GPU](#)
 - [Gradcheck mechanics](#)
 - [HIP \(ROCm\) semantics](#)
 - [Features for large-scale deployments](#)
 - [LibTorch Stable ABI](#)
 - [LocalTensor Tutorial: Single-Process SPMD Debugging](#)
 - [MKLDNN backend](#)
 - [Modules](#)
 - [MPS backend](#)
 - [Multiprocessing best practices](#)
 - [Numerical accuracy](#)
 - [Out Notes](#)
 - [Reproducibility](#)
 - [Serialization semantics](#)
 - [TensorIterator \(Python\)](#)
 - [Windows FAQ](#)
- [Community](#)
 - [PyTorch Governance | Build + CI](#)
 - [PyTorch Contribution Guide](#)
 - [PyTorch Design Philosophy](#)
 - [PyTorch Governance | Mechanics](#)
 - [PyTorch Governance | Maintainers](#)
- [Tutorials](#)

[Go to pytorch.org](#)

Ctr1+K

- [X](#)
- [GitHub](#)
- [PyTorch Forum](#)
- [PyPi](#)



- [Install PyTorch](#)
- [User Guide](#)
 - [PyTorch Main Components](#)
 - [torch.compiler](#)
 - [torch.export](#)
 - [Developer Notes](#)
 - [Accelerator Integration](#)
- [Reference API](#)
 - [torch](#)
 - [torch.nn](#)
 - [torch.nn.functional](#)
 - [torch.Tensor](#)
 - [Tensor Attributes](#)
 - [Tensor Views](#)
 - [Automatic Mixed Precision package - torch.amp](#)
 - [Automatic differentiation package - torch.autograd](#)
 - [torch.library](#)
 - [torch.accelerator](#)
 - [torch.cpu](#)
 - [torch.cuda](#)
 - [Understanding CUDA Memory Usage](#)
 - [torch.mps](#)
 - [torch.xpu](#)
 - [torch.mtia](#)
 - [torch.mtia.memory](#)
 - [torch.mtia.mtia_graph](#)
 - [Meta device](#)
 - [torch.backends](#)
 - [torch.export](#)
 - [Distributed communication package - torch.distributed](#)
 - [torch.distributed.tensor](#)
 - [Generic Join Context Manager](#)
 - [Torch Distributed Elastic](#)
 - [FullyShardedDataParallel](#)
 - [torch.distributed.fsdp.fully_shard](#)
 - [Tensor Parallelism - torch.distributed.tensor.parallel](#)
 - [Distributed Optimizers](#)
 - [Pipeline Parallelism](#)
 - [PyTorch Symmetric Memory](#)
 - [Distributed Checkpoint - torch.distributed.checkpoint](#)
 - [Probability distributions - torch.distributions](#)
 - [torch.compiler API reference](#)
 - [torch.fft](#)
 - [torch.func](#)
 - [torch.futures](#)
 - [torch.fx](#)
 - [torch.fx.experimental](#)
 - [torch.hub](#)
 - [torch.linalg](#)
 - [torch.monitor](#)
 - [torch.signal](#)
 - [torch.special](#)
 - [torch.overrides](#)
 - [torch.nativert](#)
 - [torch.package](#)
 - [torch.profiler](#)

- [torch.nn.init](#)
- [torch.nn.attention](#)
- [torch.onnx](#)
- [torch.optim](#)
- [Complex Numbers](#)
- [DDP Communication Hooks](#)
- [Quantization](#)
- [Distributed RPC Framework](#)
- [torch.random](#)
- [torch.masked](#)
- [torch.nested](#)
- [torch.Size](#)
- [torch.sparse](#)
- [torch.Storage](#)
- [torch.testing](#)
- [torch.utils](#)
- [Benchmark Utils - torch.utils.benchmark](#)
- [torch.utils.checkpoint](#)
- [torch.utils.cpp_extension](#)
- [torch.utils.data](#)
- [torch.utils.deterministic](#)
- [JIT Utils - torch.utils.jit](#)
- [torch.utils.dlpack](#)
- [torch.utils.mobile_optimizer](#)
- [torch.utils.model_zoo](#)
- [torch.utils.tensorboard](#)
- [torch.utils.module_tracker](#)
- [Type Info](#)
- [torch.config](#)
- [torch._future](#)
- [torch._logging](#)
- [Torch Environment Variables](#)
- [Developer Notes](#)
 - [Automatic Mixed Precision examples](#)
 - [Autograd mechanics](#)
 - [Broadcasting semantics](#)
 - [CPU threading and TorchScript inference](#)
 - [CUDA semantics](#)
 - [PyTorch Custom Operators Landing Page](#)
 - [Distributed Data Parallel](#)
 - [Extending PyTorch](#)
 - [Extending torch.func with autograd.Function](#)
 - [Frequently Asked Questions](#)
 - [Getting Started on Intel GPU](#)
 - [Gradcheck mechanics](#)
 - [HIP \(ROCm\) semantics](#)
 - [Features for large-scale deployments](#)
 - [LibTorch Stable ABI](#)
 - [LocalTensor Tutorial: Single-Process SPMD Debugging](#)
 - [MKLDNN backend](#)
 - [Modules](#)
 - [MPS backend](#)
 - [Multiprocessing best practices](#)
 - [Numerical accuracy](#)
 - [Out Notes](#)
 - [Reproducibility](#)
 - [Serialization semantics](#)
 - [TensorIterator \(Python\)](#)
 - [Windows FAQ](#)
- [Community](#)
 - [PyTorch Governance | Build + CI](#)
 - [PyTorch Contribution Guide](#)
 - [PyTorch Design Philosophy](#)
 - [PyTorch Governance | Mechanics](#)
 - [PyTorch Governance | Maintainers](#)
- [Tutorials](#)

[Go to pytorch.org](https://pytorch.org) Ctr1+K

- [X](#)
- [GitHub](#)
- [PyTorch Forum](#)
- [PyPi](#)

Section Navigation

- [C++](#)

Python API

- [torch](#)
- [Aliases in torch.nn](#)
- [torch.nn](#)
 - ▼
 - [Buffer](#)
 - [Parameter](#)
 - [UninitializedParameter](#)
 - [UninitializedBuffer](#)
 - [Module](#)
 - [Sequential](#)
 - [ModuleList](#)
 - [ModuleDict](#)
 - [ParameterList](#)
 - [ParameterDict](#)
 - [torch.nn.modules.module.register_module_forward_pre_hook](#)
 - [torch.nn.modules.module.register_module_forward_hook](#)
 - [torch.nn.modules.module.register_module_backward_hook](#)
 - [torch.nn.modules.module.register_module_full_backward_pre_hook](#)
 - [torch.nn.modules.module.register_module_full_backward_hook](#)
 - [torch.nn.modules.module.register_module_buffer_registration_hook](#)
 - [torch.nn.modules.module.register_module_module_registration_hook](#)
 - [torch.nn.modules.module.register_module_parameter_registration_hook](#)
 - [Conv1d](#)
 - [Conv2d](#)
 - [Conv3d](#)
 - [ConvTranspose1d](#)
 - [ConvTranspose2d](#)
 - [ConvTranspose3d](#)
 - [LazyConv1d](#)
 - [LazyConv2d](#)
 - [LazyConv3d](#)
 - [LazyConvTranspose1d](#)
 - [LazyConvTranspose2d](#)
 - [LazyConvTranspose3d](#)
 - [Unfold](#)
 - [Fold](#)
 - [MaxPool1d](#)
 - [MaxPool2d](#)
 - [MaxPool3d](#)
 - [MaxUnpool1d](#)
 - [MaxUnpool2d](#)
 - [MaxUnpool3d](#)
 - [AvgPool1d](#)
 - [AvgPool2d](#)
 - [AvgPool3d](#)
 - [FractionalMaxPool2d](#)
 - [FractionalMaxPool3d](#)
 - [LPPool1d](#)
 - [LPPool2d](#)
 - [LPPool3d](#)
 - [AdaptiveMaxPool1d](#)

- [AdaptiveMaxPool2d](#)
- [AdaptiveMaxPool3d](#)
- [AdaptiveAvgPool1d](#)
- [AdaptiveAvgPool2d](#)
- [AdaptiveAvgPool3d](#)
- [ReflectionPad1d](#)
- [ReflectionPad2d](#)
- [ReflectionPad3d](#)
- [ReplicationPad1d](#)
- [ReplicationPad2d](#)
- [ReplicationPad3d](#)
- [ZeroPad1d](#)
- [ZeroPad2d](#)
- [ZeroPad3d](#)
- [ConstantPad1d](#)
- [ConstantPad2d](#)
- [ConstantPad3d](#)
- [CircularPad1d](#)
- [CircularPad2d](#)
- [CircularPad3d](#)
- [ELU](#)
- [Hardshrink](#)
- [Hardsigmoid](#)
- [Hardtanh](#)
- [Hardswish](#)
- [LeakyReLU](#)
- [LogSigmoid](#)
- [MultiheadAttention](#)
- [PReLU](#)
- [ReLU](#)
- [ReLU6](#)
- [RReLU](#)
- [SELU](#)
- [CELU](#)
- [GELU](#)
- [Sigmoid](#)
- [SiLU](#)
- [Mish](#)
- [Softplus](#)
- [Softshrink](#)
- [Softsign](#)
- [Tanh](#)
- [Tanhshrink](#)
- [Threshold](#)
- [GLU](#)
- [Softmin](#)
- [Softmax](#)
- [Softmax2d](#)
- [LogSoftmax](#)
- [AdaptiveLogSoftmaxWithLoss](#)
- [BatchNorm1d](#)
- [BatchNorm2d](#)
- [BatchNorm3d](#)
- [LazyBatchNorm1d](#)
- [LazyBatchNorm2d](#)
- [LazyBatchNorm3d](#)
- [GroupNorm](#)
- [SyncBatchNorm](#)
- [InstanceNorm1d](#)
- [InstanceNorm2d](#)
- [InstanceNorm3d](#)
- [LazyInstanceNorm1d](#)
- [LazyInstanceNorm2d](#)
- [LazyInstanceNorm3d](#)
- [LayerNorm](#)
- [LocalResponseNorm](#)

- [RMSNorm](#)
- [RNNBase](#)
- [RNN](#)
- [LSTM](#)
- [GRU](#)
- [RNNCell](#)
- [LSTMCell](#)
- [GRUCell](#)
- [Transformer](#)
- [TransformerEncoder](#)
- [TransformerDecoder](#)
- [TransformerEncoderLayer](#)
- [TransformerDecoderLayer](#)
- [Identity](#)
- [Linear](#)
- [Bilinear](#)
- [LazyLinear](#)
- [Dropout](#)
- [Dropout1d](#)
- [Dropout2d](#)
- [Dropout3d](#)
- [AlphaDropout](#)
- [FeatureAlphaDropout](#)
- [Embedding](#)
- [EmbeddingBag](#)
- [CosineSimilarity](#)
- [PairwiseDistance](#)
- [L1Loss](#)
- [MSELoss](#)
- [CrossEntropyLoss](#)
- [LinearCrossEntropyLoss](#)
- [LinearCrossEntropyOptions](#)
- [CTCLoss](#)
- [NLLLoss](#)
- [PoissonNLLLoss](#)
- [GaussianNLLLoss](#)
- [KLDivLoss](#)
- [BCELoss](#)
- [BCEWithLogitsLoss](#)
- [MarginRankingLoss](#)
- [HingeEmbeddingLoss](#)
- [MultiLabelMarginLoss](#)
- [HuberLoss](#)
- [SmoothL1Loss](#)
- [SoftMarginLoss](#)
- [MultiLabelSoftMarginLoss](#)
- [CosineEmbeddingLoss](#)
- [MultiMarginLoss](#)
- [TripletMarginLoss](#)
- [TripletMarginWithDistanceLoss](#)
- [PixelShuffle](#)
- [PixelUnshuffle](#)
- [Upsample](#)
- [UpsamplingNearest2d](#)
- [UpsamplingBilinear2d](#)
- [ChannelShuffle](#)
- [DataParallel](#)
- [DistributedDataParallel](#)
- [torch.nn.utils.clip_grad_norm](#)
- [torch.nn.utils.clip_grad_norm](#)
- [torch.nn.utils.clip_grad_value](#)
- [torch.nn.utils.get_total_norm](#)
- [torch.nn.utils.clip_grads_with_norm](#)
- [torch.nn.utils.parameters_to_vector](#)
- [torch.nn.utils.vector_to_parameters](#)
- [torch.nn.utils.fuse_conv_bn_eval](#)

- [torch.nn.utils.fuse_conv_bn_weights](#)
- [torch.nn.utils.fuse_linear_bn_eval](#)
- [torch.nn.utils.fuse_linear_bn_weights](#)
- [torch.nn.utils.convert_conv2d_weight_memory_format](#)
- [torch.nn.utils.convert_conv3d_weight_memory_format](#)
- [torch.nn.utils.weight_norm](#)
- [torch.nn.utils.remove_weight_norm](#)
- [torch.nn.utils.spectral_norm](#)
- [torch.nn.utils.remove_spectral_norm](#)
- [torch.nn.utils.skip_init](#)
- [BasePruningMethod](#)
- [PruningContainer](#)
- [Identity](#)
- [RandomUnstructured](#)
- [L1Unstructured](#)
- [RandomStructured](#)
- [LnStructured](#)
- [CustomFromMask](#)
- [torch.nn.utils.prune.identity](#)
- [torch.nn.utils.prune.random_unstructured](#)
- [torch.nn.utils.prune.l1_unstructured](#)
- [torch.nn.utils.prune.random_structured](#)
- [torch.nn.utils.prune.ln_structured](#)
- [torch.nn.utils.prune.global_unstructured](#)
- [torch.nn.utils.prune.custom_from_mask](#)
- [torch.nn.utils.prune.remove](#)
- [torch.nn.utils.prune.is_pruned](#)
- [torch.nn.utils.parametrizations.orthogonal](#)
- [torch.nn.utils.parametrizations.weight_norm](#)
- [torch.nn.utils.parametrizations.spectral_norm](#)
- [torch.nn.utils.parametrize.register_parametrization](#)
- [torch.nn.utils.parametrize.remove_parametrizations](#)
- [torch.nn.utils.parametrize.cached](#)
- [torch.nn.utils.parametrize.is_parametrized](#)
- [torch.nn.utils.parametrize.transfer_parametrizations_and_params](#)
- [torch.nn.utils.parametrize.type_before_parametrizations](#)
- [ParametrizationList](#)
- [torch.nn.utils.stateless.functional_call](#)
- [PackedSequence](#)
- [torch.nn.utils.rnn.pack_padded_sequence](#)
- [torch.nn.utils.rnn.pad_packed_sequence](#)
- [torch.nn.utils.rnn.pad_sequence](#)
- [torch.nn.utils.rnn.pack_sequence](#)
- [torch.nn.utils.rnn.unpack_sequence](#)
- [torch.nn.utils.rnn.unpad_sequence](#)
- [torch.nn.utils.rnn.invert_permutation](#)
- [torch.nn.parameter.is_lazy](#)
- [torch.nn.factory_kwargs](#)
- [Flatten](#)
- [Unflatten](#)
- [LazyModuleMixin](#)
- [torch.nn.functional](#)
- [torch.Tensor](#)
- [Tensor Attributes](#)
- [Tensor Views](#)
- [torch.amp](#)
- [torch.autograd](#)
- [torch.library](#)
- [torch.accelerator](#)
- [torch.cpu](#)
- [torch.cuda](#)
- [torch.cuda.memory](#)
- [torch.mps](#)
- [torch.xpu](#)
- [torch.mtia](#)
- [torch.mtia.memory](#)

- [torch.mtia.mtia_graph](#)
- [Meta device](#)
- [torch.backends](#)
- [torch.export](#)
- [torch.distributed](#)
- [torch.distributed.tensor](#)
- [torch.distributed.algorithms.join](#)
- [torch.distributed.elastic](#)
- [torch.distributed.fsdp](#)
- [torch.distributed.fsdp.fully_shard](#)
- [torch.distributed.tensor.parallel](#)
- [torch.distributed.optim](#)
- [torch.distributed.pipelining](#)
- [torch.distributed._symmetric_memory](#)
- [torch.distributed.checkpoint](#)
- [torch.distributions](#)
- [torch.compiler](#)
- [torch.fft](#)
- [torch.func](#)
- [torch.futures](#)
- [torch.fx](#)
- [torch.fx.experimental](#)
- [torch.hub](#)
- [torch.linalg](#)
- [torch.monitor](#)
- [torch.signal](#)
- [torch.special](#)
- [torch.overrides](#)
- [torch.nativert](#)
- [torch.package](#)
- [torch.profiler](#)
- [torch.nn.init](#)
- [torch.nn.attention](#)
- [torch.onnx](#)
- [torch.optim](#)
- [Complex Numbers](#)
- [DDP Communication Hooks](#)
- [Quantization](#)
- [Distributed RPC Framework](#)
- [torch.random](#)
- [torch.masked](#)
- [torch.nested](#)
- [torch.Size](#)
- [torch.sparse](#)
- [torch.Storage](#)
- [torch.testing](#)
- [torch.utils](#)
- [Benchmark Utils - torch.utils.benchmark](#)
- [torch.utils.checkpoint](#)
- [torch.utils.cpp_extension](#)
- [torch.utils.data](#)
- [torch.utils.deterministic](#)
- [JIT Utils - torch.utils.jit](#)
- [torch.utils.dlpack](#)
- [torch.utils.mobile_optimizer](#)
- [torch.utils.model_zoo](#)
- [torch.utils.tensorboard](#)
- [torch.utils.module_tracker](#)
- [Type Info](#)
- [torch.__config__](#)
- [torch.__future__](#)
- [torch._logging](#)
- [Torch Environment Variables](#)

-
- [Reference API](#)
- [torch.nn](#)
- ConvTranspose1d

Rate this Page

★ ★ ★ ★ ★

ConvTranspose1d#

`class torch.nn.ConvTranspose1d(in_channels, out_channels, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True, dilation=1, padding_mode='zeros', device=None, dtype=None)`[\[source\]](#)#

Applies a 1D transposed convolution operator over an input image composed of several input planes.

This module can be seen as the gradient of `Conv1d` with respect to its input. It is also known as a fractionally-strided convolution or a deconvolution (although it is not an actual deconvolution operation as it does not compute a true inverse of convolution). For more information, see the visualizations [here](#) and the [Deconvolutional Networks](#) paper.

This module supports [TensorFloat32](#).

On certain ROCm devices, when using float16 inputs this module will use [different precision](#) for backward.

- `stride` controls the stride for the cross-correlation.
- `padding` controls the amount of implicit zero padding on both sides for $\text{dilation} * (\text{kernel_size} - 1) - \text{padding}$ number of points. See note below for details.
- `output_padding` controls the additional size added to one side of the output shape. See note below for details.
- `dilation` controls the spacing between the kernel points; also known as the à trous algorithm. It is harder to describe, but the link [here](#) has a nice visualization of what `dilation` does.
- `groups` controls the connections between inputs and outputs. `in_channels` and `out_channels` must both be divisible by `groups`. For example,
 - At `groups=1`, all inputs are convolved to all outputs.
 - At `groups=2`, the operation becomes equivalent to having two conv layers side by side, each seeing half the input channels and producing half the output channels, and both subsequently concatenated.
 - At `groups=in_channels`, each input channel is convolved with its own set of filters (of size $\frac{\text{out_channels}}{\text{in_channels}}$).

Note

The `padding` argument effectively adds $\text{dilation} * (\text{kernel_size} - 1) - \text{padding}$ amount of zero padding to both sides of the input. This is set so that when a [Conv1d](#) and a [ConvTranspose1d](#) are initialized with same parameters, they are inverses of each other in regard to the input and output shapes. However, when `stride > 1`, [Conv1d](#) maps multiple input shapes to the same output shape. `output_padding` is provided to resolve this ambiguity by effectively increasing the calculated output shape on one side. Note that `output_padding` is only used to find output shape, but does not actually add zero-padding to output.

Note

In some circumstances when using the CUDA backend with CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. Please see the notes on [Reproducibility](#) for background.

Parameters:

- **in_channels** ([int](#)) – Number of channels in the input image
- **out_channels** ([int](#)) – Number of channels produced by the convolution
- **kernel_size** ([int](#) or [tuple](#)) – Size of the convolving kernel

- **stride** (*int or tuple, optional*) – Stride of the convolution. Default: 1
- **padding** (*int or tuple, optional*) – dilation * (kernel_size - 1) - padding zero-padding will be added to both sides of the input. Default: 0
- **output_padding** (*int or tuple, optional*) – Additional size added to one side of the output shape. Default: 0
- **groups** (*int, optional*) – Number of blocked connections from input channels to output channels. Default: 1
- **bias** (*bool, optional*) – If True, adds a learnable bias to the output. Default: True
- **dilation** (*int or tuple, optional*) – Spacing between kernel elements. Default: 1

Shape:

- Input: (N, C_{in}, L_{in}) or (C_{in}, L_{in})
- Output: (N, C_{out}, L_{out}) or (C_{out}, L_{out}) , where

$$L_{out} = (L_{in} - 1) \times \text{stride} - 2 \times \text{padding} + \text{dilation} \times (\text{kernel_size} - 1) + \text{output_padding} + 1$$

Variables:

- **weight** (*Tensor*) – the learnable weights of the module of shape $(\text{in_channels}, \frac{\text{out_channels}}{\text{groups}}, \text{kernel_size})$. The values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{\text{groups}}{C_{out} * \text{kernel_size}}$
- **bias** (*Tensor*) – the learnable bias of the module of shape (out_channels) . If bias is True, then the values of these weights are sampled from $\mathcal{U}(-\sqrt{k}, \sqrt{k})$ where $k = \frac{\text{groups}}{C_{out} * \text{kernel_size}}$

Examples:

```
>>> # With square kernels and equal stride
>>> m = nn.ConvTranspose1d(16, 33, 3, stride=2)
>>> input = torch.randn(20, 16, 50)
>>> output = m(input)
>>> # exact output size can be also specified as an argument
>>> input = torch.randn(1, 16, 12)
>>> downsample = nn.Conv1d(16, 16, 3, stride=2, padding=1)
>>> upsample = nn.ConvTranspose1d(16, 16, 3, stride=2, padding=1)
>>> h = downsample(input)
>>> h.size()
torch.Size([1, 16, 6])
>>> output = upsample(h, output_size=input.size())
>>> output.size()
torch.Size([1, 16, 12])
```

Rate this Page

★★★★★

[previous](#)

[Conv3d](#)

[next](#)

[ConvTranspose2d](#)

Built with the [PyData Sphinx Theme](#) 0.15.4.

[previous](#)

[Conv3d](#)

[next](#)

[ConvTranspose2d](#)

On this page

- [ConvTranspose1d](#)

[Show Source](#)

PyTorch Libraries

[ExecuTorch](#)

[Helion](#)

[torchao](#)

[kineto](#)

[torchtitan](#)

[TorchRL](#)

[torchvision](#)

[torchaudio](#)

[tensordict](#)

[PyTorch on XLA Devices](#)

Docs

Access comprehensive developer documentation for PyTorch

[View Docs](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers

[View Tutorials](#)

Resources

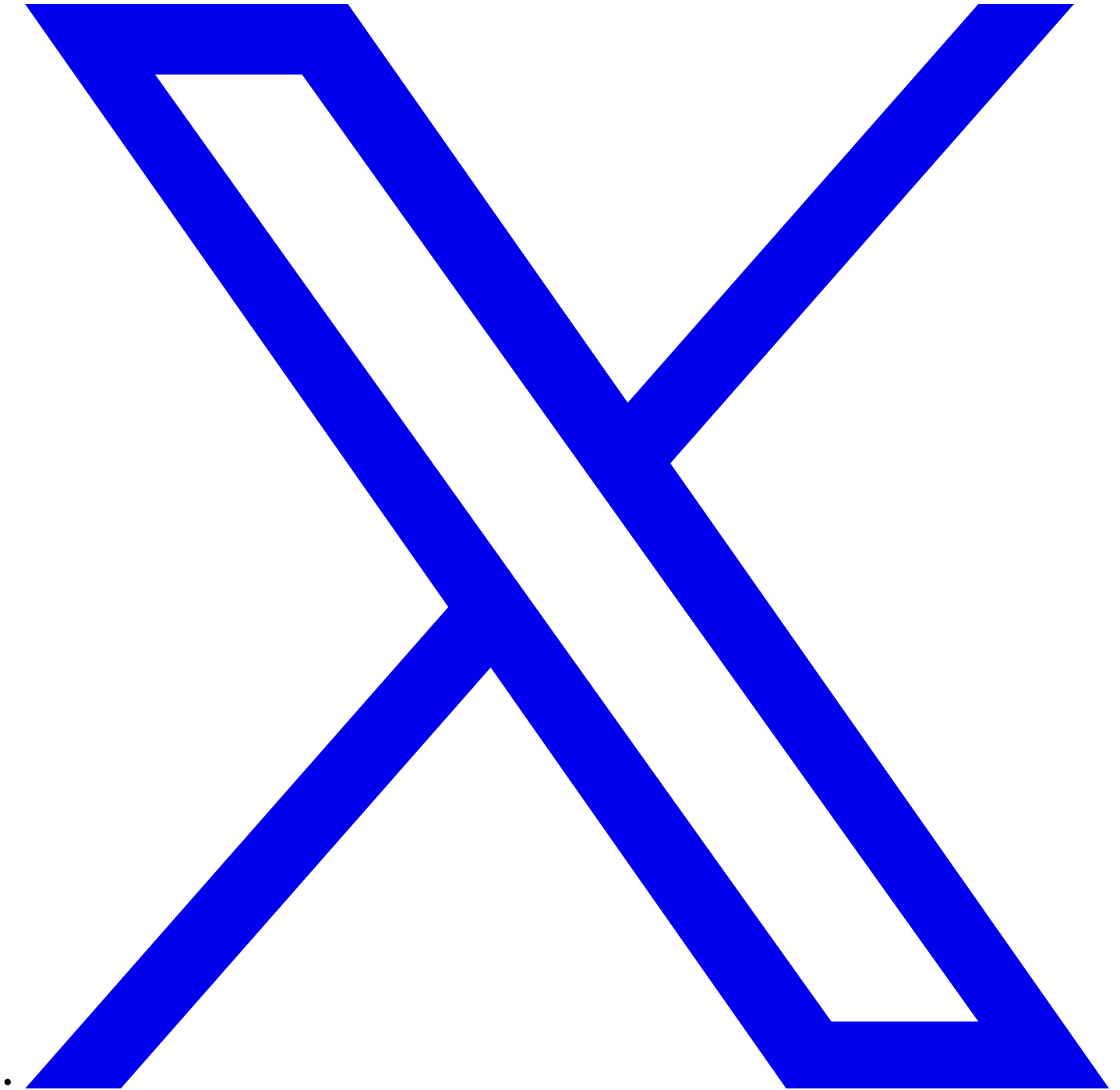
Find development resources and get your questions answered

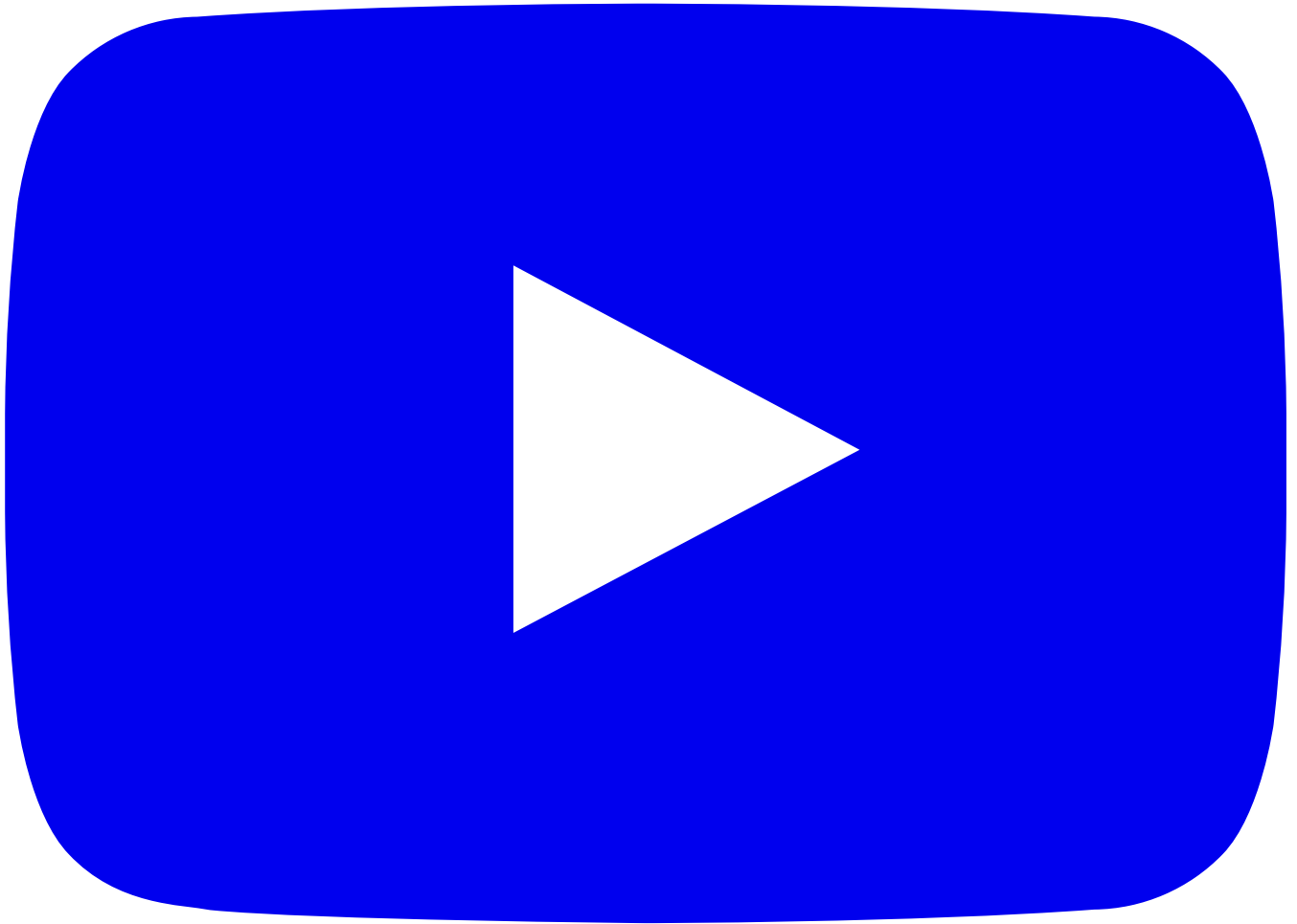
[View Resources](#)

Stay in touch for updates, event info, and the latest news

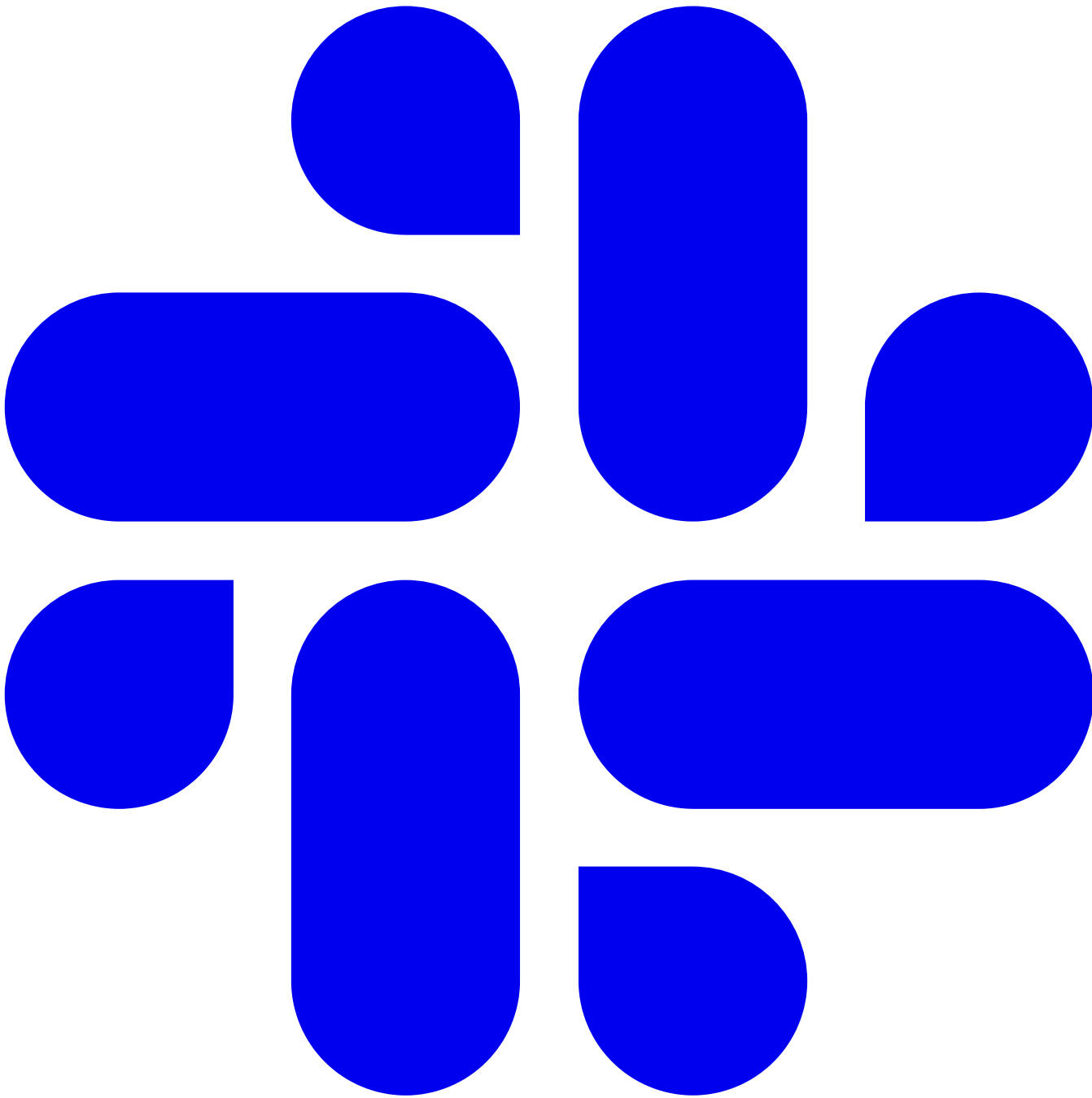
By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. [Privacy Policy](#).













•

© PyTorch. Copyright © The Linux Foundation®. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies](#) page. [Trademark Usage](#). [Privacy Policy](#).

To analyze traffic and optimize your experience, we serve cookies on this site. By clicking or navigating, you agree to allow our usage of cookies. As the current maintainers of this site, Facebook's Cookies Policy applies. Learn more, including about available controls: [Cookies Policy](#).

□

© Copyright PyTorch Contributors.

Created using [Sphinx](#) 7.2.6.

Built with the [PyData Sphinx Theme](#) 0.15.4.